КОНЦЕПТ ИСПОЛЬЗОВАНИЯ: ЭКРАНЫ РОБОТА И ДОСТУП К НИМ

> **Документ:** docs/RUN_AND_ACCESS.md **Версия:** 1.2 · **Дата:** 2026-10 **Связанные документы:** MAIN_SCREEN.md · GUI.md · SERVICE_CONSOLE.md · SERIAL_PROTOCOL.md · SOFTWARE_ARCHITECTURE.md

Документ отвечает на вопросы: **какие экраны есть у робота**, **где какой показывается**, **как устроен сервер и какие у него соединения**, **что такое PIN-код отсека и как он устанавливается**, и **кто чем пользуется** в смене и на соревнованиях.

Главное в одной строке: **основной экран — полноэкранно на роботе** (веб-страница или нативное приложение; там же вводят PIN), а **инженерный пульт — удалённо** с ноутбука или планшета. Сервер борта отдаёт веб-файлы и JSON API; нативный экран обращается только к API и сам рисует интерфейс.

---

1. Два экрана — два уровня доступа

Экран	Файл и адрес	Где показывается	Кому	Что показывает
Основной экран робота	веб: `main.html`, адрес `/`; нативно: `gui/native_display.py` / собранное приложение	**полноэкранно на дисплее робота**; API обновляется примерно каждые 300 мс	оператор смены, получатель груза, судьи	4 двигателя (угол, об/мин, температура, состояние), заряд АКБ, PIN и состояние отсека, режим и связь
Инженерный пульт	`index.html`, адреса `/index.html` и `/console`	**удалённо**: ноутбук или планшет инженера в сети робота	инженер-наладчик	карта SLAM, миссия, программа доставки, телеметрия модулей, сервисное окно (замок, ручные команды, статистика), журнал

PIN-код вводят **на самом роботе** — на основном экране его дисплея; с этого же экрана открывается и закрывается грузовой отсек борта (сервер выполняет команду замка, см. §3). Удалённо пульт видит то же состояние замка и тот же журнал доступа.

Логика разделения:

- на основном экране **нет** опасных действий — только состояние и открытие отсека по PIN-коду (это нужно и оператору, и получателю груза);
- всё, что меняет поведение робота (ручные команды модулям, режим «стенд», сброс статистики, программы доставки), вынесено на инженерный пульт;
- оба экрана отдаёт **один процесс** — `gui/backend.py`, поэтому они всегда видят одни и те же данные и одно состояние замка.

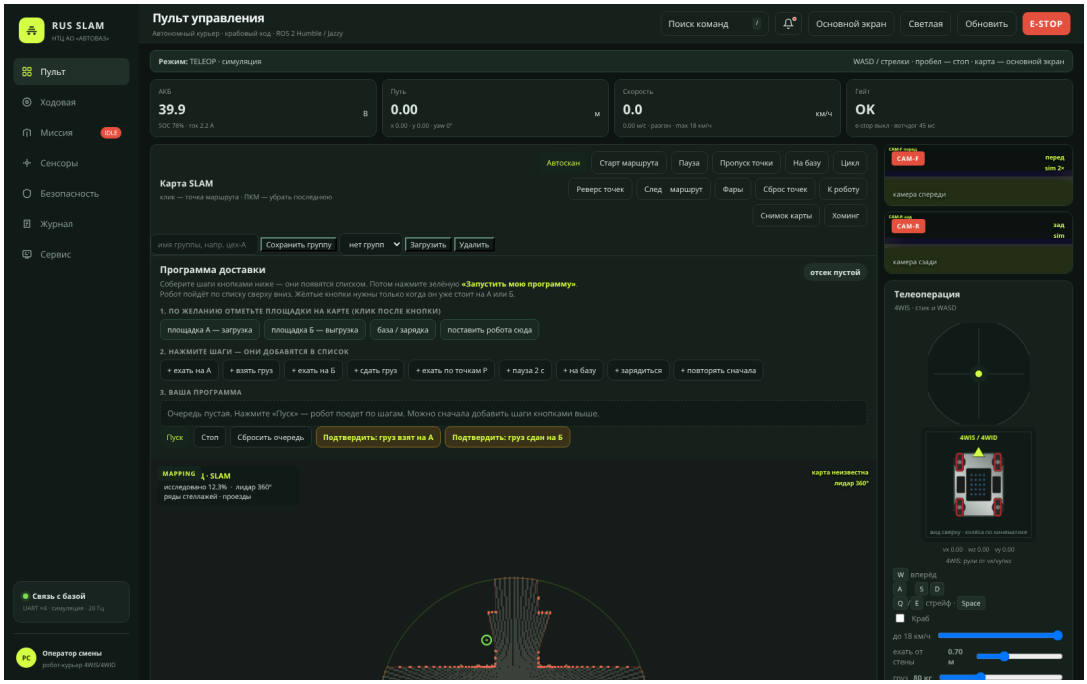
1.1. Скриншоты обоих экранов

Сняты с работающего сервера: `python3 gui/backend.py --port 8080 --source sim`.

Основной экран робота — он же киоск на дисплее борта, адрес `/`:



Инженерный пульт — удалённый доступ, адрес `/console`:



Остальные вкладки пульта («Ходовая», «Миссия», «Сенсоры», «Безопасность», «Журнал», «Сервис») — в [GUI.md](), §1.1.

---

2. Как это устроено: один процесс, один порт

Весь «сервер» — это **один процесс Python на борту робота**:

```
python3 gui/backend.py --port 8080        # или короткий синоним: python3 gui/serve.py
```

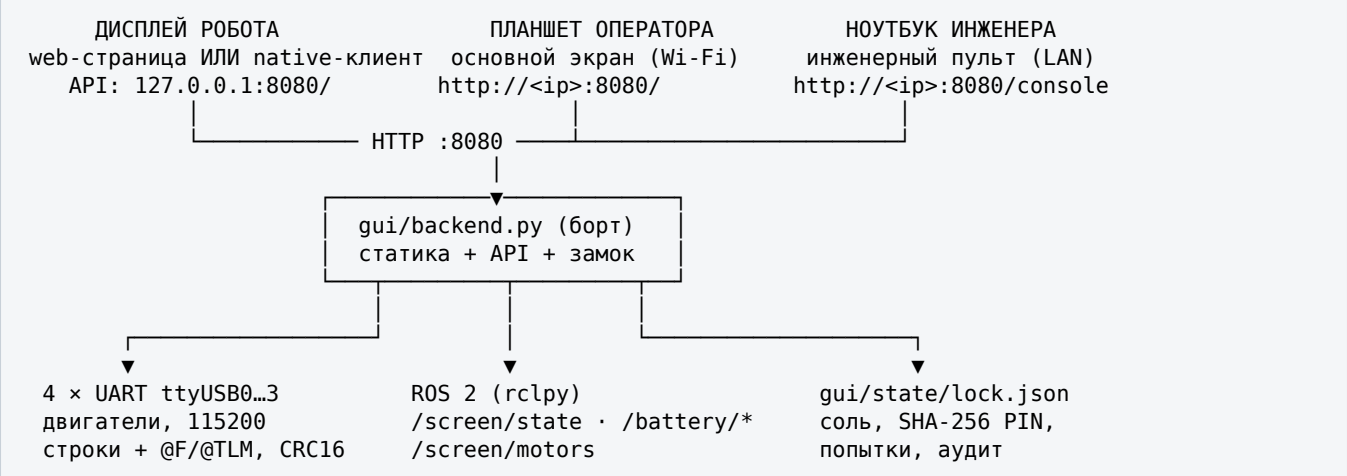
Он обслуживает веб-вариант и общую логику:

1. **отдаёт страницы** — HTML/CSS/JS из папки `gui/` для браузерных экранов;
2. **держит JSON API** — `/api/state`, `/api/lock/*`, `/api/audit`, `/api/health`;
3. **ведёт замок** — проверяет PIN, считает попытки, пишет журнал доступа.

Браузерный экран использует HTML/CSS/JS. Нативное приложение (`gui/native_display.py`) работает без браузера: рисует окно средствами Tkinter и читает/изменяет данные только

через JSON API. Оба клиента получают одинаковое состояние замка и те же показатели. Ноутбук или планшет инженера по-прежнему открывает веб-пульт.

Сервер слушает `0.0.0.0:8080`, то есть доступен и на самом роботе (`127.0.0.1`), и всем в сети (`http://<ip-робота>:8080`). **Сервер сам ничего не открывает и не запускает** — дисплейный клиент запускается отдельным сервисом (§4.1). Больше ни одного открытого порта для экранов не требуется.



2.1. Что и куда подключается

Направление	Интерфейс	Что передаётся
дисплей робота → борт	HTTP JSON API, `127.0.0.1:8080`	`GET /api/state` (300 мс), `POST /api/lock/open` — веб- и нативный клиент используют общий PIN и состояние замка
пульт инженера → борт	HTTP по Wi-Fi/LAN, `<ip-робота>:8080`	`GET /api/state`, `/api/audit`, `POST /api/lock/*` — то же состояние замка и тот же журнал
борт → платы колёс	4 × USB `ttyUSB*`/`ttyACM*`, 115200 8N1 (pyserial)	текстовые команды (`w`, `v`, `t`, `b`, `k`, `q`, `ph`, `zq`/`zc`); телеметрия `@F` ~30 мс и `@TLM` 200 мс с CRC16 (протокол — `docs/SERIAL_PROTOCOL.md`)
борт ↔ ROS 2	DDS через `rclpy`	подписки `/screen/state` (JSON), `/battery/voltage`, `/battery/current`, `/screen/motors`
борт → замок отсека	команда ОС из окружения	`RUS_SLAM_LOCK_CMD` (открыть), `RUS_SLAM_LOCK_CMD_CLOSE` (закрыть): реле, соленоид, GPIO
борт → диск	файл `gui/state/lock.json`	соль, хеш PIN, счётчик неудач, блокировка, журнал доступа
борт → дисплей робота	браузерный или нативный процесс	по умолчанию **ничего не открывается**; веб-страницу показывает `rus-slam-display.service`, нативное приложение — `rus-slam-native-display.service`; флаги `--kiosk` / `--open` относятся к веб-варианту

Что из этого обязательного: **сервер** `backend.py`; дисплейный клиент, UART, ROS 2 и железо замка — отдельные подключения. Для нативного клиента нужны Tkinter и графическая сессия; его можно собрать в отдельный бинарник. По умолчанию источник данных — `--source sim` (модель): API и оба варианта основного экрана работают без железа, поэтому систему можно показывать на стенде. Переключатели источника:

```
python3 gui/backend.py --source sim                                    # модель (по умолчанию)
python3 gui/backend.py --source serial \
    --ports /dev/ttyUSB0,/dev/ttyUSB1,/dev/ttyUSB2,/dev/ttyUSB3        # реальные модули
python3 gui/backend.py --source ros                                    # бортовые топики ROS 2
python3 gui/backend.py --source auto                                   # ROS 2, а без него — модель
```

Если для выбранного источника нет библиотеки (`pyserial` для `serial`, `rclpy` для `ros`) или порт не открывается, борт **не падает**: в консоли появляется предупреждение, модуль/источник переходит в модель.

---

3. PIN-код грузового отсека: что это и как устанавливается

PIN-код — это цифровой пароль (4–8 цифр), которым открывается грузовой отсек. Он защищает груз от посторонних: по регламенту конкурса за это дают до 5 баллов (критерий «безопасность груза» — «нет доступа посторонним»). Заводское значение — `2580`, его печатает `--help` и `/api/health`.

3.1. Как PIN хранится

В открытом виде PIN нигде не лежит:

```
gui/state/lock.json
{
  "version": 1,
  "salt":     "<32 hex-символа, случайные 16 байт>",
  "pinHash":  "SHA-256(salt + \":\" + PIN)",
  "fails":    0,          // неудачных попыток подряд
  "lockUntil": 0,         // до какого времени ввод заблокирован
  "audit":    [ … ],      // журнал доступа с хеш-цепочкой (последние 200 записей)
  "changedAt": …
}
```

Файл создаётся сам при первом запуске сервера и лежит в `.gitignore` (в репозиторий не попадает). Соль у каждого робота своя, поэтому одинаковые PIN на двух роботах дают разные хеши, а по украденному файлу PIN не подобрать перебором таблицы.

3.2. Три способа установить PIN

Способ	Команда	Когда действует
Заводской при первом старте	`python3 gui/backend.py --pin 1234`	только если файла `lock.json` ещё нет (или после сброса)
Смена из сервисного окна пульта	кнопка смены PIN либо `POST /api/lock/pin` с телом `{"current":"2580","new":"1234"}`	нужен текущий PIN; новый — 4–8 цифр, повтор должен совпасть
Сброс к заводскому	`python3 gui/backend.py --reset-pin`	удаляет файл: следующий запуск создаёт его заново с PIN `--pin` (по умолчанию `2580`)

После успешной смены: генерируется новая соль, старый хеш становится недействительным, отсек закрывается, в журнал пишется `pin_change`.

3.3. Защита от подбора

- **5 неудачных попыток** подряд → блокировка ввода на **30 секунд** (`--max-attempts`, `--lock-sec` меняют пороги);
- счётчик попыток и время блокировки **хранит сервер** (в `lock.json`), а не страница: перезагрузка экрана, планшета или браузера блокировку не снимает;
- каждая попытка — запись в журнал: успех, отказ с остатком попыток, блокировка, смена PIN. Записи связаны хеш-цепочкой: `GET /api/audit` возвращает `integrity.ok = false`, если журнал правили руками;
- ограничение честное: интерфейс работает по **HTTP без шифрования**, поэтому сеть робота должна быть локальной и доверенной (цех, стенд). Развитие — HTTPS и отдельный пароль инженера на пульт (см. §5).

3.4. Проверка PIN через API

```
# неверный PIN → 403, остаток попыток
curl -s -X POST http://127.0.0.1:8080/api/lock/open \
     -H 'Content-Type: application/json' -d '{"pin":"1111"}'
# {"ok": false, "reason": "wrong", "attemptsLeft": 4, …}

# верный PIN → 200, отсек открыт (и команда железу, если задан RUS_SLAM_LOCK_CMD)
curl -s -X POST http://127.0.0.1:8080/api/lock/open \
     -H 'Content-Type: application/json' -d '{"pin":"2580"}'

# смена PIN
curl -s -X POST http://127.0.0.1:8080/api/lock/pin \
     -H 'Content-Type: application/json' -d '{"current":"2580","new":"1234"}'
```

---

4. Как попадать на экраны

4.1. Основной экран на роботе: браузер или нативное приложение

Сервер борта окон не открывает. Клиент дисплея запускается отдельным сервисом, поэтому backend можно перезапускать независимо. Выберите **один** вариант.

Браузерный экран:

```
sudo systemctl enable --now rus-slam-server     # gui/backend.py на 0.0.0.0:8080
sudo systemctl enable --now rus-slam-display    # браузер без рамок на /
```

Нативный экран без HTML/браузера (сначала соберите бинарник на Ubuntu/Linux):

```
sudo apt install -y python3-tk python3-venv
cd /opt/rus_slam/gui && ./tools/build-native-linux.sh
sudo systemctl disable --now rus-slam-display
sudo systemctl enable --now rus-slam-native-display
```

Нативное приложение (`gui/native_display.py` либо `gui/dist/RUS-SLAM-Display`) самостоятельно рисует интерфейс с Tk Canvas и опрашивает `/api/state` каждые 300 мс; PIN и команды замка отправляются в `/api/lock/*`. Без связи с API экран показывает ошибку и блокирует действия с замком. Собранный запускной файл получается через PyInstaller. Windows `.exe` собирается на Windows командой `gui/tools/build-native-windows.bat`; для удалённого робота задайте `--url` (например, `RUS-SLAM-Display.exe --url http://192.168.1.10:8080`).

Установочные команды, параметры пользователя и дисплея описаны в `deploy/README.md`. Если у робота нет дисплея, оба сервиса показа не нужны — основной экран открывают на планшете, а пульт по `/console`.

Разовый веб-запуск остаётся доступен:

```
python3 gui/backend.py --kiosk      # сервер + страница в браузере без рамок
python3 gui/backend.py --open       # сервер + веб-страница в обычном окне
python3 gui/backend.py              # только сервер (окон не открывает)
python3 gui/native_display.py --url http://127.0.0.1:8080 --windowed
```

В обоих вариантах ввод PIN-кода выполняется на дисплее, отсек открывается **на самом роботе**, а сервер вызывает команду замка (см. §3.1).

4.2. Инженерный пульт — удалённо

Пульт не запускают на роботе: в него заходят с ноутбука или планшета в сети робота:

```
http://<ip-робота>:8080/console        инженерный пульт (адрес-синоним: /index.html)
```

IP печатается в консоли сервера при запуске (строка «инженерный пульт: …» / «с планшета: …»). Пульт видит те же данные и тот же замок, что и экран на роботе: смена PIN и журнал доступа — общие.

4.3. Переход между экранами

Откуда → куда	Как
основной экран робота → инженерный пульт	веб: ссылка в подвале; нативный экран не содержит перехода на пульт; откройте `/console` отдельно на устройстве инженера
инженерный пульт → основной экран	кнопка **«Основной экран»** в шапке пульта или адрес `/`
напрямую	`/` — основной экран · `/index.html`, `/console` — пульт

4.4. С планшета или ноутбука — всё то же самое

Робот и устройство в одной сети (Wi-Fi цеха/стенда): `http://<ip-робота>:8080/` отдаёт основной экран любому браузеру, `…/console` — инженерный пульт. Так получатель груза может посмотреть состояние отсека со своего телефона, а инженер — работать с пультом, не подходя к роботу.

4.5. Остановка и перезапуск

- сервер: `Ctrl+C` в терминале, либо `sudo systemctl restart rus-slam-server`;
- веб-экран: `sudo systemctl restart rus-slam-display`; нативный экран: `sudo systemctl restart rus-slam-native-display`;
- перезапуск не теряет данные: PIN и журнал доступа лежат в `gui/state/lock.json`, статистика пульта — в `localStorage` браузера.

---

5. Роли и защита доступа

Роль	Что может	Как ограничивается
Оператор смены	открыть/закрыть отсек, читать состояние	PIN-код отсека, 5 попыток → блокировка 30 с
Инженер-наладчик	ручные команды модулям, статистика, настройки	доступ к пульту по сети (см. §3.3), режим «стенд», интерлоки E-STOP
Получатель груза	только забрать груз	видит основной экран, но не может запустить движение

Что уже защищено технически:

- PIN-код хранится как `SHA-256(salt + PIN)`, заводской `2580` меняется из сервисного окна или через API `POST /api/lock/pin` (см. §3);
- журнал доступа — с хеш-цепочкой: подделка или удаление записи обнаруживаются (`GET /api/audit` возвращает `integrity.ok = false`);
- открытие отсека в движении не предусмотрено: команды движения живут на инженерном пульте, а он в режиме миссии блокирует ручной ввод.

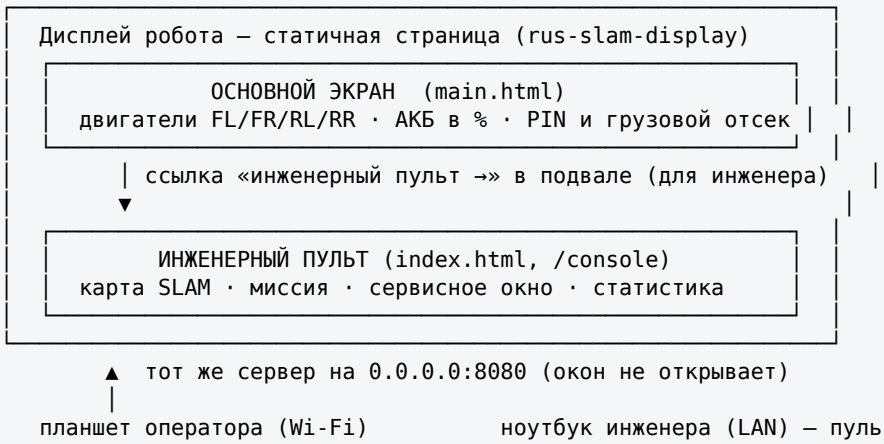
Развитие (документированное, не реализовано): вход в инженерный пульт по отдельному паролю инженера и/или ограничение доступа по IP, HTTPS, разграничение ролей «оператор/инженер/администратор», журнал входов.

---

6. Сценарии смены

Сценарий	Действия	Экран
Выдача груза получателю	робот приехал → получатель набирает PIN 2580 → «Открыть отсек» → забирает груз → «Закрыть отсек»	основной
Загрузка на складе	оператор открывает отсек по PIN, укладывает груз, закрывает; на пульте — «Подтвердить: груз взят»	основной + пульт
Контроль состояния	взгляд на экран: заряд в процентах, температура и состояние каждого модуля, связь	основной
Низкий заряд	при < 40 % подпись «низкий заряд», при < 33,5 В — «аварийный порог… движение запрещено»; оператор отправляет робота на базу	основной
Отладка модулей	инженер: пульт → сервисное окно → режим «стенд» → ручные команды FL/FR/RL/RR с контролем кадра UART	инженерный пульт
Разбор инцидента	пульт → сервисное окно → журнал доступа (аудит) и журнал сервисных действий; выгрузка CSV/JSON/печать отчёта	инженерный пульт
Демонстрация судьям	основной экран на дисплее робота: видно заряд, модули, отсек и время; смена PIN-кода с удалённого пульта на глазах у судей подтверждает защиту груза	основной + пульт

7. Раскладка на роботе



Один процесс backend.py = один источник данных для всех клиентов: сколько бы устройств ни открыли экраны, показания и состояние замка будут одинаковыми.

8. Быстрая проверка доступа

```
# 1. запустить бэкенд (стенд, без робота)
python3 gui/backend.py --port 8080

# 2. убедиться, что API жив
curl -s http://127.0.0.1:8080/api/health
curl -s http://127.0.0.1:8080/api/state | head -c 300

# 3. отсек: неверный и верный PIN
curl -s -X POST http://127.0.0.1:8080/api/lock/open -H 'Content-Type: application/json' -d '{"pin":"1111"}'
curl -s -X POST http://127.0.0.1:8080/api/lock/open -H 'Content-Type: application/json' -d '{"pin":"2580"}'

# 4. проверить, что киоск отдаётся с корня
curl -sI http://127.0.0.1:8080/ | head -3
```

Автотесты доступа и API:

```
python3 gui/tests/backend.test.py                                    # 40 проверок API и замка
NODE_PATH=/tmp/harness/node_modules node gui/tests/main.api.test.js  # 20: экран ↔ бэкенд
```

9. Лицензия и авторы

MIT, команда RUS SLAM, НТЦ АО «АВТОВАЗ», конкурс «Первые в профессии», задание № 2 «Автономное роботизированное устройство для внутренней логистики».